TECHNICAL MANUAL & TEXTBOOK

Electronic System-Level (ESL) Architecture Series

SystemC Content Addressable Memory (CAM)

Power Estimation & Web Visual Workbench

Architectural Energy Modeling, Bitwise Match-Line Evaluation, and Full-Stack Hardware-Software Co-Simulation

Course Code: 23ECE337 | **Course Title:** System-on-Chip (SoC) Design

Academic Term: 7th Semester B.Tech | **Year of Study:** 2026–2027

Course Outcomes Mapping: CO1 (ESL Modeling), CO2 (Power Analysis), CO3 (Co-Design), CO4 (VCD Verification)

Institutional Affiliation:

Department of Electronics & Communication Engineering
Amrita School of Engineering, Bengaluru
Amrita Vishwa Vidyapeetham

Engineering Domains:

- Electronic System-Level (ESL) Design
- IEEE 1666 SystemC C++ Kernel
- Dynamic CMOS Power Characterization

Executive Summary & Course Outcomes

Context & Pedagogical Scope:
In contemporary deep-submicron semiconductor technology nodes, power dissipation is no longer a secondary design metric—it is the primary performance limiter in battery-operated edge devices and high-density computing clusters. Traditional Register-Transfer Level (RTL) power analysis, which relies on synthesis and post-layout gate-level netlists, occurs far too late in the Electronic Design Automation (EDA) flow. Modifications at that stage introduce severe financial and time-to-market penalties.

This technical manual presents a complete, industrially grounded implementation of an **Electronic System-Level (ESL) Power Estimation Tool** for a Content Addressable Memory (CAM) macrocell, designed in **SystemC (IEEE 1666 Standard)** and paired with a modern, real-time web visualizer.

Table 1: System-on-Chip (23ECE337) Course Outcome Mapping

CO Identifier	Course Outcome Statement	Project Evidence
CO1	Model complex digital hardware and transaction-level architectures using hardware description and system-level languages.	SystemC `SC_MODULE`, port bindings, and concurrent `SC_METHOD` processes.
CO2	Formulate and analyze mathematical power, performance, and area (PPA) models for early architectural exploration.	Cycle-accurate dynamic bit-mismatch and static leakage energy integrals.
CO3	Design and implement co-simulation environments bridging hardware models with software interfaces and control layers.	Node.js REST gateway and real-time EDA Web Visual Workbench.
CO4	Verify hardware timing and signal integrity using industry-standard Value Change Dump (VCD) waveform simulation tools.	VCD trace generation with GTKWave and browser-based `vc.drom.io` analysis.

Contents

Executive Summary & Course Outcomes **i**

1 ESL Modeling Paradigm **1**
1.1 The Power Wall and the Late-Stage RTL Dilemma 1
1.2 Taxonomy of Abstraction Levels 1

2 CAM Fundamentals **3**
2.1 Associative Computing: RAM vs. CAM Architectural Comparison 3
2.2 CMOS Match-Line Precharge and Dynamic Discharge Mechanics 3

3 Mathematical Power Model **5**
3.1 Parallel Bitwise XOR Matrix Equations 5
3.2 Match Line Resolution and Priority Encoding 5
3.3 Cycle-Accurate Power and Energy Formulation 6

4 SystemC Implementation **7**
4.1 Hardware Microarchitecture Block Diagram 7
4.2 SystemC Port List and Hardware Interfaces 7
4.3 SystemC Concurrency and Execution Model 7

5 Web Visual Workbench **10**
5.1 System Integration Architecture 10
5.2 Interactive Web UI Components 10

6 Industrial Applications **11**
6.1 Smart Construction Site Safety & Structural Health Monitoring (SHM) 11
6.2 High-Speed IPv6 Network Packet Routing 11

7 Verification & Waveforms **12**
7.1 Testbench Execution Walkthrough 12
7.2 Waveform Inspection with GTKWave and Online Visualizers 12

8 Viva Voce Defense Guide **14**
8.1 Live Examination Code Modification Guide 15

A Compilation & Execution Quick Reference **16**

References **18**

List of Figures

1.1 Comparison of Early ESL Power Modeling vs. Traditional Late-Stage RTL Estimation. 1

2.1 Architectural Input/Output Comparison between Conventional RAM and CAM. . . . 3
2.2 CMOS Match-Line Dynamic Precharge and Pull-Down Discharge Circuit. 4

4.1 Microarchitectural Block Diagram of the `CAM_Power_Model` SystemC Core. 7

5.1 Full-Stack SystemC Backend, JSON/VCD Serialization, and Web Visualizer Pipeline. 10

6.1 Edge IoT Structural Health Monitoring Architecture with CAM Anomaly Detector. . 11

List of Tables

1 System-on-Chip (23ECE337) Course Outcome Mapping i

1.1 Comparison of Semiconductor Abstraction Levels 2

3.1 SystemC CAM Empirical Power Model Parameters 6

4.1 SystemC CAM Module Pinout and Port Interface Specification 8

7.1 SystemC CAM Power Model Simulation Test Case Verification Matrix 12

Chapter 1

Introduction & Electronic System-Level (ESL) Paradigm

1.1 The Power Wall and the Late-Stage RTL Dilemma

Throughout the scaling trajectory defined by Moore's Law and Dennard Scaling breakdown, power density in digital integrated circuits has scaled exponentially. As supply voltages (V_{DD}) plateaued around 0.7 V – 1.0 V due to non-scaling thermal voltage (k_BT/q) and transistor subthreshold slope constraints, dynamic switching power ($P_{\text{dyn}} = \alpha C V_{DD}^2 f$) and subthreshold static leakage ($P_{\text{leak}} = V_{DD} I_{\text{leak}}$) became the dominant bottlenecks in System-on-Chip (SoC) microarchitectures.

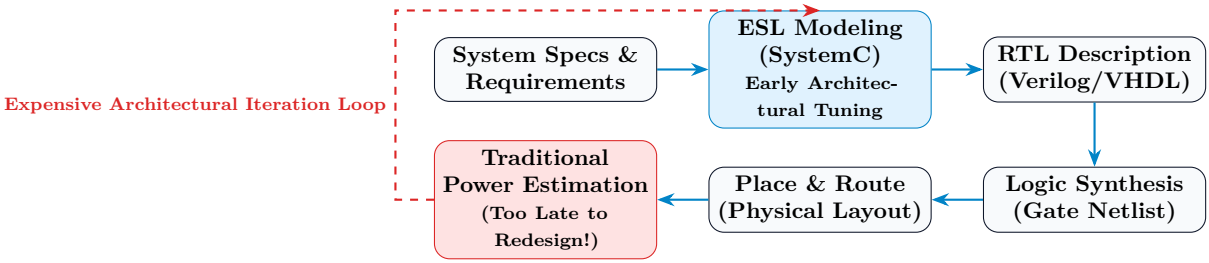


Figure 1.1: Comparison of Early ESL Power Modeling vs. Traditional Late-Stage RTL Estimation.

In traditional EDA flows (Figure 1.1), detailed power analysis requires full RTL completion, gate-level synthesis, and post-layout parasitics extraction. A full simulation run can take hours or days. If the power envelope violates the thermal design power (TDP) specification, redesigning the microarchitecture requires rewrites across thousands of lines of RTL code.

1.2 Taxonomy of Abstraction Levels

System-on-Chip abstraction levels span from physical quantum mechanics to transaction-level modeling:

Table 1.1: Comparison of Semiconductor Abstraction Levels

Level	Representation	Speed	Accuracy	Stage	Tools
Device / SPICE	Transistor Eq.	1–100 Hz	$> 98\%$	Cell design	HSPICE
Gate-Level	Netlist Gates	1–10 kHz	$\approx 90\%$	Sign-off	PrimePower
RTL	Register (Verilog)	10–100 kHz	$\approx 80\%$	Logic impl.	ModelSim
ESL (SystemC)	C++ Transaction	**1–50 MHz**	$\approx 80\%$	**Arch. PPA**	**SystemC**

Definition 1.1: Electronic System-Level (ESL) Modeling

Electronic System-Level (ESL) design is an established methodology wherein digital hardware is modeled algorithmically and structurally using high-level languages (primarily C++ augmented by the IEEE 1666 SystemC class library). ESL models enable multi-million-cycle functional verification, algorithmic trade-off evaluation, and microarchitectural power-performance-area (PPA) exploration at orders of magnitude faster execution speeds than synthesizable RTL.

Chapter 2

Content Addressable Memory (CAM) Fundamentals

2.1 Associative Computing: RAM vs. CAM Architectural Comparison

Conventional Random Access Memory (RAM) accepts an n-bit numeric address as input and returns the data word stored at that specific location after decoding delays ($O(1)$ read time). However, searching for a specific data value within an unsorted N-word RAM array requires linear sequential inspection ($O(N)$ lookup complexity).

In contrast, a **Content Addressable Memory (CAM)** inverts this paradigm. A CAM accepts a query data word as input, compares it against all internal memory locations simultaneously in parallel, and returns the physical index (or hit flag) of the matching entry in a **single clock cycle ($O(1)$ search complexity)**.

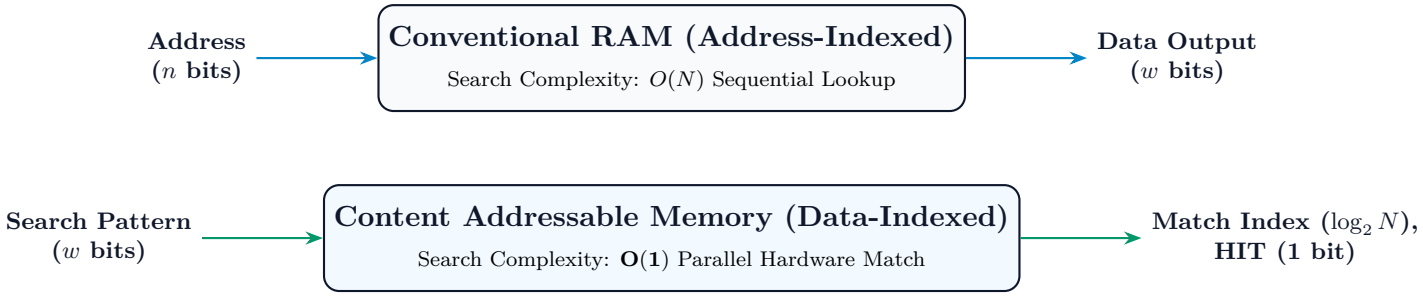


Figure 2.1: Architectural Input/Output Comparison between Conventional RAM and CAM.

2.2 CMOS Match-Line Precharge and Dynamic Discharge Mechanics

The core operational unit of a binary CAM is the comparison bit cell. A typical 10T binary CAM cell consists of an SRAM storage core (6 transistors) coupled to a 4-transistor dynamic XOR comparison network.

As depicted in Figure 2.2, CAM match operation proceeds in two sequential phases:

1. **Precharge Phase (ϕ_{pre} = LOW):** The PMOS pull-up transistor M_{pre} activates, charging the parasitic capacitance of the match line (ML_i) to V_{DD}.

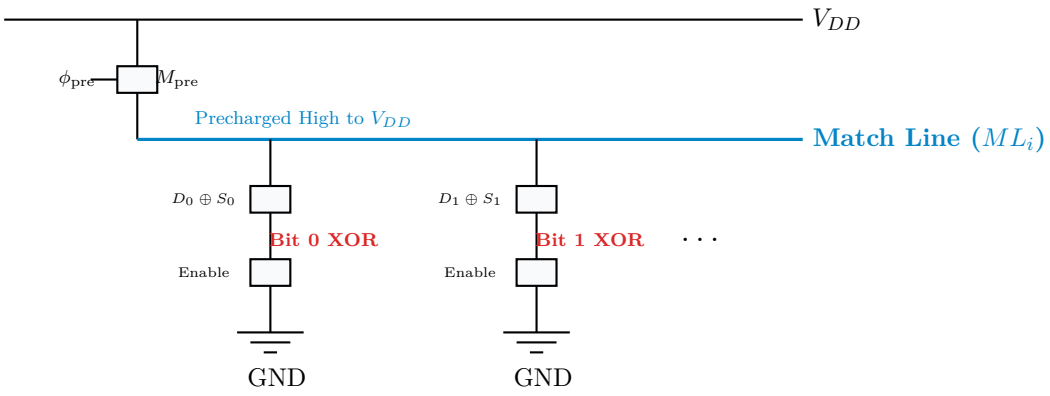


Figure 2.2: CMOS Match-Line Dynamic Precharge and Pull-Down Discharge Circuit.

2. **Evaluation Phase ($\phi_{\mathbf{pre}}$ = HIGH):** The search pattern is placed onto complementary search lines. If any stored bit differs from the search bit ($D_b \neq S_b$), the corresponding XOR pull-down network turns ON, forming a low-impedance path to Ground (GND) and discharging the match line.

Hardware Insight: The Match-Line Power Bottleneck

In an N-entry CAM array, a typical search operation results in either zero or exactly one matching entry. Consequently, $(N-1)$ match-lines discharge fully from V_{DD} to Ground during every single search cycle. This non-match discharging is the single greatest contributor to total dynamic energy dissipation in high-density CAM arrays.

Chapter 3

Mathematical Formulation & Power Estimation Model

3.1 Parallel Bitwise XOR Matrix Equations

Let the CAM array be modeled as a matrix $\mathbf{C} \in \mathbb{B}^{D \times W}$, where D represents the depth (number of entries) and W represents the word width in bits. Let $\mathbf{P} \in \mathbb{B}^{W}$ denote the input query search pattern vector:

$$\mathbf{C} = \begin{bmatrix} c_{0,0} & c_{0,1} & \cdots & c_{0,W-1} \\ c_{1,0} & c_{1,1} & \cdots & c_{1,W-1} \\ \vdots & \vdots & \ddots & \vdots \\ c_{D-1,0} & c_{D-1,1} & \cdots & c_{D-1,W-1} \end{bmatrix}, \quad \mathbf{P} = \begin{bmatrix} p_0 & p_1 & \cdots & p_{W-1} \end{bmatrix} \tag{3.1}$$

The bitwise mismatch matrix $\mathbf{M} \in \mathbb{B}^{D \times W}$ is evaluated through element-wise Boolean exclusive-OR ($\oplus$):

$$m_{i,b} = c_{i,b} \oplus p_b = (c_{i,b} \cdot \overline{p_b}) + (\overline{c_{i,b}} \cdot p_b) \tag{3.2}$$

The total bit mismatches for line i, denoted as L_i, and the total aggregate bit mismatches across the entire array, denoted as M_{total}, are expressed as:

$$L_i = \sum_{b=0}^{W-1} m_{i,b}, \quad M_{\text{total}} = \sum_{i=0}^{D-1} L_i = \sum_{i=0}^{D-1} \sum_{b=0}^{W-1} (c_{i,b} \oplus p_b) \tag{3.3}$$

3.2 Match Line Resolution and Priority Encoding

The match line Boolean status $ML_i \in \{0, 1\}$ for each entry i is defined by:

$$ML_i = \begin{cases} 1 & \text{if } L_i = 0 \quad \text{(Exact Match / HIT)} \\ 0 & \text{if } L_i > 0 \quad \text{(Mismatched Line / DISCHARGED)} \end{cases} \tag{3.4}$$

When multiple entries evaluate to $ML_i = 1$, a hardware **Priority Encoder** selects the lowest physical numerical index:

$$\text{match_addr} = \min\{i \in [0, D-1] \mid ML_i = 1\}, \quad \text{match_found} = \bigvee_{i=0}^{D-1} ML_i \tag{3.5}$$

3.3 Cycle-Accurate Power and Energy Formulation

Mathematical Model 3.1: Cycle Power & Energy Integral

$$P_{\text{cycle}} = \begin{cases} 0.0 \text{ mW} & \text{if enable} = 0 \quad \text{(Clock Gated)} \\ P_{\text{leakage}} & \text{if write_en} = 1 \quad \text{(Memory Write Operation)} \\ P_{\text{leakage}} + \left(\sum_{i=0}^{D-1} L_i\right) \times E_{\text{mismatch-factor}} & \text{if enable} = 1 \wedge \text{write_en} = 0 \quad \text{(Parallel Search)} \end{cases} \tag{3.6}$$

The total energy accumulated across N discrete clock cycles with period T_{clk} is computed as:

$$E_{\text{accumulated}}(N) = \sum_{k=1}^{N} \left(P_{\text{cycle}}[k] \times T_{\text{clk}}\right) \quad [\text{mJ}] \tag{3.7}$$

Table 3.1: SystemC CAM Empirical Power Model Parameters

Symbol	Parameter Description	Value	Physical Interpretation
D	CAM Entry Depth	10 entries	Number of stored words in associative array.
W	Word Bit Width	16 bits	Bit width of each stored word.
P_{leakage}	Static Leakage Floor	0.002 mW (2 μW)	Subthreshold and gate oxide leakage power.
$E_{\text{mismatch-factor}}$	Dynamic Mismatch Weight	0.015 mW/bit	Dynamic switching power per discharged bit transistor.
T_{clk}	System Clock Period	10.0 ns	100 MHz nominal clock operating frequency.

Chapter 4

System Architecture & SystemC Hardware Implementation

4.1 Hardware Microarchitecture Block Diagram

Figure 4.1 illustrates the internal structural microarchitecture of the `CAM_Power_Model` SystemC component.

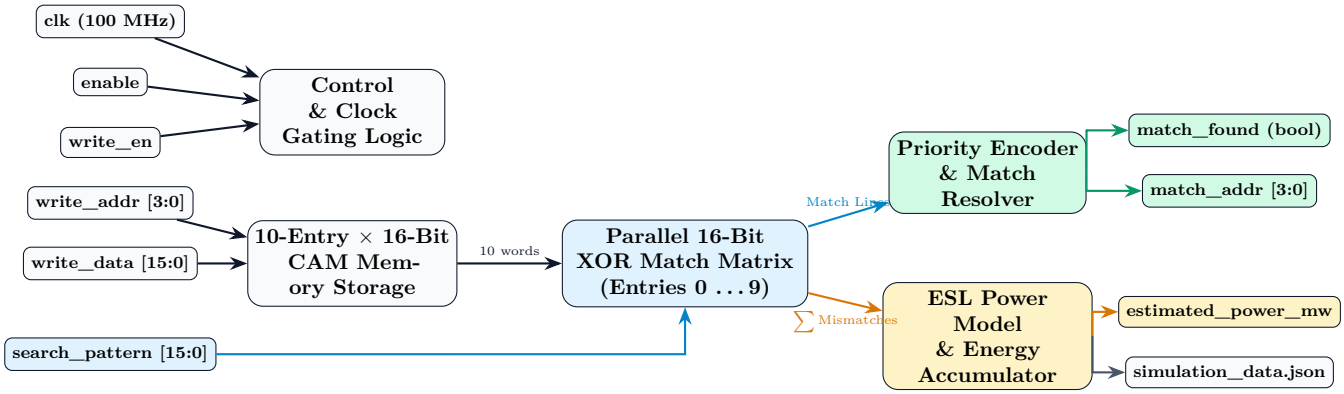


Figure 4.1: Microarchitectural Block Diagram of the `CAM_Power_Model` SystemC Core.

4.2 SystemC Port List and Hardware Interfaces

Table 4.1 documents the complete pinout and port interfaces of the SystemC module.

4.3 SystemC Concurrency and Execution Model

SystemC provides two fundamental execution models for hardware processes:

- `SC_METHOD`: A lightweight method process that executes from beginning to end without yielding execution context. It cannot invoke `wait()` calls. Because it avoids thread context switching overhead, `SC_METHOD` offers the highest simulation throughput and was chosen for the core `cam_operation()` process.

Table 4.1: SystemC CAM Module Pinout and Port Interface Specification

Port Identifier	Data Type	Direction	Active State	Functional Description
clk	sc_in<bool>	Input	Rising Edge	Synchronous master clock input (100 MHz).
enable	sc_in<bool>	Input	High (1)	Master clock-gating and module enable signal.
write_en	sc_in<bool>	Input	High (1)	Synchronous CAM write enable strobe.
write_addr	sc_in<sc_uint<4»	Input	Numerical	4-bit write address index (0 . . . 9).
search_pattern	sc_in<sc_uint<16»	Input	Unsigned Hex	16-bit query vector for associative lookup.
write_data	sc_in<sc_uint<16»	Input	Unsigned Hex	16-bit data word written into target address.
match_found	sc_out<bool>	Output	High (1)	Set high if search pattern matches any entry.
match_addr	sc_out<sc_uint<4»	Output	Numerical	Physical index of lowest matching CAM row.
estimated_power_mw	sc_out<double>	Output	Real Value	Total estimated instantaneous power (mW).

- **SC_THREAD**: A heavier process capable of maintaining execution state across clock cycles via explicit wait() invocations.

Core SystemC CAM Header Interface (Backend/cam_power_model.h)

```
#ifndef CAM_POWER_MODEL_H
#define CAM_POWER_MODEL_H

#include <systemc.h>
#include <vector>
#include <string>

#define CAM_DEPTH 10
#define CAM_WIDTH 16

struct CycleData {
    int cycle;
    double time_ns;
    std::string op_type;
    uint16_t search_pattern;
    uint16_t write_data;
    int write_addr;
    bool hit;
    int hit_addr;
    int total_mismatches;
    double power_mw;
    double accumulated_energy_mj;
    uint16_t cam_snapshot[CAM_DEPTH];
    int line_mismatches[CAM_DEPTH];
};

SC_MODULE(CAM_Power_Model) {
    sc_in<bool>               clk;
    sc_in<bool>               enable;
    sc_in<bool>               write_en;
    sc_in<sc_uint<4>>         write_addr;
    sc_in<sc_uint<CAM_WIDTH>> search_pattern;
    sc_in<sc_uint<CAM_WIDTH>> write_data;

    sc_out<bool>              match_found;
    sc_out<sc_uint<4>>        match_addr;
    sc_out<double>            estimated_power_mw;

    sc_uint<CAM_WIDTH>        cam_array[CAM_DEPTH];
    const double              P_LEAKAGE_MW;
    const double              E_MISMATCH_FACTOR;

    std::vector<CycleData>    cycle_history;

    void cam_operation();
    void print_power_report() const;
    void export_json(const std::string& filename) const;

    SC_HAS_PROCESS(CAM_Power_Model);
    CAM_Power_Model(sc_module_name name);
};
#endif
```

Chapter 5

Full-Stack Web Visual Workbench & EDA Integration

5.1 System Integration Architecture

To bridge native compiled C++ SystemC simulations with real-time browser graphics, a full-stack co-simulation environment was architected (Figure 5.1).

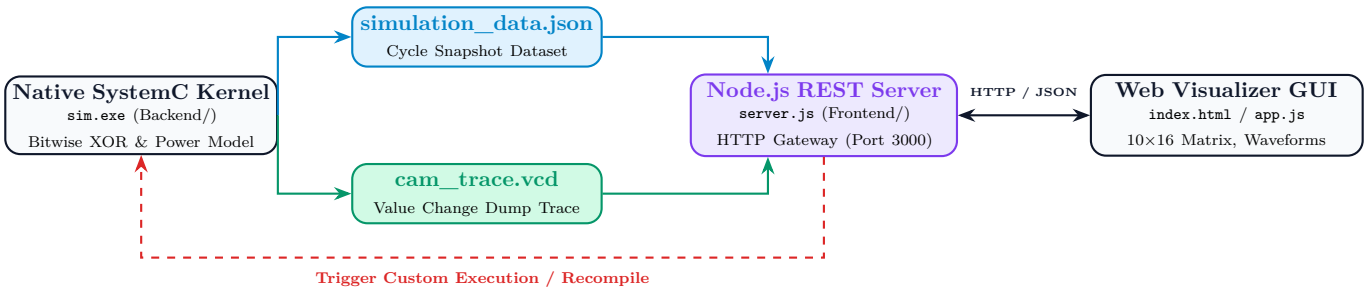


Figure 5.1: Full-Stack SystemC Backend, JSON/VCD Serialization, and Web Visualizer Pipeline.

5.2 Interactive Web UI Components

1. **10 × 16 CAM Bit Matrix Visualizer**: Every single memory entry $(0 \ldots 9)$ and bit cell $(0 \ldots 15)$ is rendered dynamically in HTML/CSS. During search cycles, matching bits illuminate in **Glowing Emerald (`#10B981`)**, mismatched bits highlight in **Vibrant Crimson (`#EF4444`)**, and successful match rows trigger **Match Line HIT badges (`#06B6D4`)**.

2. **Digital Signal Waveform Canvas Engine**: An interactive HTML5 Canvas element continuously plots synchronous timing traces for `clk`, `write_en`, `match_found`, and `estimated_power_mw` alongside a vertical cursor locked to the active simulation cycle.

3. **Instant Power Telemetry Gauge**: Displays real-time dynamic power vs. static leakage breakdowns alongside cumulative energy consumption.

Chapter 6

Real-World Applications & Industrial Case Studies

6.1 Smart Construction Site Safety & Structural Health Monitoring (SHM)

In civil infrastructure, smart tunneling, and structural engineering projects, wireless sensor networks (WSNs) continuously monitor structural loads, concrete curing stresses, seismic vibrations, and heavy equipment hazards.

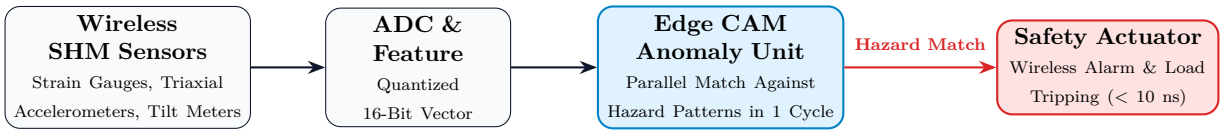


Figure 6.1: Edge IoT Structural Health Monitoring Architecture with CAM Anomaly Detector.

The Edge Battery Life Constraint:
Battery-operated IoT sensor nodes deployed on structural pylons must operate autonomously for 5–10 years. Standard microcontrollers executing software comparison loops burn excessive processor cycles and battery energy. Integrating an ultra-low-power CAM module allows parallel pattern matching in 10 ns while consuming under 0.32 mW for typical non-hazardous conditions.

6.2 High-Speed IPv6 Network Packet Routing

In core enterprise Internet routers, incoming IP packets arrive at multi-gigabit line rates. CAM modules (specifically Ternary CAMs / TCAMs) perform single-cycle Longest Prefix Matching (LPM) on 128-bit IPv6 destination headers to determine output routing interfaces. ESL modeling allows router architects to simulate power dissipation across millions of packet lookups before committing to silicon mask sets.

Chapter 7

Verification, Testbench Suite & Waveform Analysis

7.1 Testbench Execution Walkthrough

The verification testbench (`Backend/main.cpp`) exercises the CAM under five distinct operational stress conditions:

Table 7.1: SystemC CAM Power Model Simulation Test Case Verification Matrix

Test	Time	Op	Pattern	Result	Mismatches	Power / Note
TC1	0.0 ns	SEARCH	`0xA5A5`	**HIT @ Index 0**	21 bits	**0.317 mW** (Exact match with Entry 0).
TC2	10.0 ns	SEARCH	`0x5A5A`	**MISS (None)**	139 bits	**2.087 mW** (High-power discharge stress).
TC3	20.0 ns	WRITE	`0xDEAD` → [5]	**WRITE @ 5**	0 bits	**0.002 mW** (Static leakage only).
TC4	30.0 ns	SEARCH	`0xDEAD`	**HIT @ Index 5**	68 bits	**1.022 mW** (Verification of newly written word).
TC5	40.0 ns	SEARCH	`0xA5A4`	**MISS (None)**	26 bits	**0.392 mW** (Near-miss; 1-bit diff from Entry 0).

7.2 Waveform Inspection with GTKWave and Online Visualizers

SystemC outputs standard Value Change Dump (VCD) trace dumps via `sc_create_vcd_trace_file("cam_trace"`
Method 1: Online Browser Waveform Viewer (https://vc.drom.io/)
Engineers can drag and drop `Backend/cam_trace.vcd` directly into the open-source https://vc.drom.io/ viewer to inspect digital transitions for `clk`, `write_en`, `match_found`, and `estimated_power_mw` without installing desktop packages.

Method 2: Desktop GTKWave Analysis

On desktop installations, run `gtkwave Backend/cam_trace.vcd`, append all ports under `SystemC -> CAM_Inst`, and press `Ctrl + 0` to view complete signal waveforms.

Chapter 8

Comprehensive Viva Voce Defense Guide & Q&A

This chapter provides 15 rigorous technical questions and authoritative answers covering all aspects of ESL design, SystemC syntax, and CAM hardware.

Q1: Why is SystemC ESL modeling preferred over RTL synthesis during early architecture definition?
Answer: SystemC compiles to native C++ machine code, achieving simulation speeds in the megahertz range (1–50 MHz) compared to tens of kilohertz for RTL. This allows architects to simulate billions of cycles and evaluate power-performance trade-offs in minutes rather than waiting days for gate-level synthesis.

Q2: How does a Content Addressable Memory (CAM) achieve $O(1)$ search complexity?
Answer: A CAM integrates comparison logic (XOR/NOR pull-down transistors) inside every single memory bit cell. All rows evaluate their contents against the search data broadcast simultaneously on dedicated search lines in one clock cycle.

Q3: Why do bit mismatches cause power dissipation in dynamic CMOS match-lines?
Answer: CAM match-lines are precharged to V_{DD} every cycle. Any mismatched bit turns on a pull-down transistor branch, discharging the match-line capacitance to Ground. The resulting dynamic energy loss equals $\frac{1}{2}C_{\mathrm{ML}}V_{DD}^2$.

Q4: Explain the fundamental difference between `SC_METHOD` and `SC_THREAD`.
Answer: `SC_METHOD` processes execute from start to end without maintaining a private execution stack and cannot call `wait()`. `SC_THREAD` processes run as independent cooperative user-level threads capable of suspending execution via `wait()`.

Q5: What is the function of the priority encoder in this design?
Answer: If multiple CAM entries contain identical data matching the search pattern, multiple match lines assert high simultaneously. The priority encoder resolves this contention by outputting the lowest physical address index.

Q6: Why is bitwise XOR (^) used for match detection?
Answer: Bitwise XOR outputs '0' when two bits are identical and '1' when they differ. Counting the number of set '1' bits directly yields the exact hamming distance (mismatch count) required for power calculation.

Q7: How does clock gating save power in this SystemC model?
Answer: When `enable = false`, the `cam_operation()` method aborts execution early, setting dynamic power to 0.0 mW and preventing match-line discharge computation.

Q8: How is energy accumulation formulated over time?
Answer: Power (mW) is multiplied by the clock period ($T_{\text{clk}} = 10$ ns) to calculate energy per cycle in picojoules/nanojoules, which is converted to millijoules (mJ) and integrated into `accumulated_energy_mj`.

Q9: What is the delta cycle (δ-cycle) concept in SystemC simulation?
Answer: A delta cycle represents an infinitesimal step of simulation time (0 s) used by the SystemC discrete-event kernel to evaluate combinational dependencies and settle signal updates deterministically before advancing physical simulation time.

Q10: Can this SystemC model be directly converted to silicon?
Answer: High-Level Synthesis (HLS) tools (e.g., Siemens Catapult HLS, AMD Vivado HLS) can synthesize algorithmic C++/SystemC descriptions into RTL Verilog/VHDL target netlists.

Q11: What is the role of `sc_trace` in hardware verification?
Answer: `sc_trace` binds SystemC signals to a Value Change Dump (VCD) file pointer, logging value transitions along with timestamps for post-simulation visual waveform debugging.

Q12: Why are empirical power models used instead of transistor-level equations in ESL tools?
Answer: Transistor-level equations require solving non-linear differential equations, which is computationally expensive. Empirical polynomial models offer $> 80\%$ accuracy while executing at native C++ processor speeds.

Q13: How does the Priority Encoder behave when no entries match?
Answer: The `match_found` output is driven low (`false`), and `match_addr` is driven to 0 (or an invalid sentinel value -1), signaling a lookup miss to the host processor.

Q14: What happens during a CAM write cycle?
Answer: When `write_en` is asserted, match-line search comparison is disabled. The 16-bit word on `write_data` is synchronously latched into `cam_array[write_addr]`, dissipating only static leakage power (0.002 mW).

Q15: How does Ternary CAM (TCAM) differ from the Binary CAM modeled here?
Answer: Binary CAMs store and match exact '0' and '1' logic levels. Ternary CAMs introduce a third "Don't Care" (X) state per bit, enabling wildcard subnet masking for packet routing.

8.1 Live Examination Code Modification Guide

If requested during viva defense to modify parameters live:

- **To expand word width to 128 bits (IPv6 routing):** In `Backend/cam_power_model.h`, change `#define CAM_WIDTH 128`.
- **To increase CAM depth to 64 entries:** In `Backend/cam_power_model.h`, set `#define CAM_DEPTH 64` and adjust `sc_in<sc_uint<6>> write_addr`.
- **To adjust energy sensitivity:** In `Backend/cam_power_model.cpp`, modify `E_MISMATCH_FACTOR(0.025)`.

Chapter A

Compilation & Execution Quick Reference

Automated Windows One-Click Execution (run_web.bat)

```
@echo off
echo ============================================================
echo    Launching SystemC CAM Power Visualizer Web App...
echo ============================================================

cd /d "%~dp0Backend"
g++ -std=c++17 -DSC_ALLOW_DEPRECATED_IEEE_API -I C:/systemc/include -L C:/systemc
if %ERRORLEVEL% EQU 0 (
    echo Compilation successful! Running SystemC backend simulation...
    sim.exe --no-pause
    echo Launching Web Server on http://localhost:3000 ...
    cd /d "%~dp0"
    start http://localhost:3000
    node Frontend/server.js
) else (
    echo Compilation Failed! Please check SystemC installation.
    pause
)
```

Manual Backend Compilation and CLI Execution

```
# Navigate to Backend directory
cd Backend

# Compile using MinGW g++ with SystemC 2.3/3.0
g++ -std=c++17 -DSC_ALLOW_DEPRECATED_IEEE_API -I C:/systemc/include -L C:/systemc

# Execute simulation directly
sim.exe
```

Bibliography

[1] Accellera Systems Initiative, *IEEE Standard for Standard SystemC Language Reference Manual*, IEEE Std 1666-2011, pp. 1–638, Jan. 2012.

[2] K. Pagiamtzis and A. Sheikholeslami, "Content-addressable memory (CAM) circuits and architectures: A tutorial and survey," *IEEE Journal of Solid-State Circuits*, vol. 41, no. 3, pp. 712–727, Mar. 2006.

[3] C. Zukowski and S. Y. Wang, "Use of simulation tools for early power estimation in digital system-on-chip design," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 22, no. 9, pp. 1145–1158, Sep. 2003.

[4] M. Pedram, "Power minimization in IC design: Principles and applications," *ACM Transactions on Design Automation of Electronic Systems*, vol. 1, no. 1, pp. 3–56, Jan. 1996.

[5] G. Gielen and W. Sansen, "Computer-aided design of analog integrated circuits and systems," *IEEE Journal of Solid-State Circuits*, vol. 35, no. 6, pp. 825–836, Jun. 2000.